Claude Code Prompt Playbook: Space Exploration Website Overhaul

How to Use This Guide

Work through each step in order. **Do not skip ahead** — each step builds on the previous one. After each prompt, review what Claude Code produces, test it, and fix any issues before moving to the next step.

Before you start, tell Claude Code about your current stack so it doesn't make wrong assumptions.

STEP 0: Context Setting (Run once at the very beginning)

When: Before anything else. This is your opening message in a fresh Claude Code session.

```
Read public/space-website-design-guide.md — this is our master design
reference for a full site overhaul. Don't make any changes yet.

Here's our current tech stack:
- [Framework: e.g., React/Next.js/Vite/etc.]
- [3D library: e.g., Three.js/react-three-fiber/none yet]
- [Database: e.g., Supabase/PostgreSQL/Firebase/JSON files]
- [Styling: e.g., Tailwind/CSS Modules/styled-components]
- [Hosting: e.g., Vercel/Netlify/GitHub Pages]

Give me a brief assessment of what needs to change architecturally
to support the design guide, and flag any tech stack conflicts.
Don't write code yet — just give me the plan.
```

Review: Read its architectural assessment. If it suggests switching frameworks or adding dependencies, decide now whether you agree before proceeding.

PHASE 1: FOUNDATION

Step 1: Design Tokens & Theme System

When: First coding step. Everything else depends on this.

```
Referring to Section 2 of public/space-website-design-guide.md,
create a complete design token system for our project:

1. CSS custom properties (or Tailwind config if we're using Tailwind)
   for all colors — backgrounds, accents, text, status colors
   (success green, failure red, anomaly amber)
2. Typography scale with the font stack from the guide —
   headings, body, monospace for data
3. Spacing scale (4px base)
4. Border radius tokens
5. Shadow/glow tokens for the space aesthetic
6. Transition/animation duration tokens
7. A prefers-reduced-motion media query that disables animations globally

Put these in a central theme file that the entire app imports from.
Use the exact hex values from the guide.
```

Review: Check that all colors exist, open the app and visually confirm the dark theme applies globally. Test contrast ratios of text colors against backgrounds using WebAIM's contrast checker.

Step 2: Core Layout Shell & Navigation

When: After the theme is in place.

```
Referring to Sections 3.1-3.3 of public/space-website-design-guide.md,
build the core layout shell:

1. A full-viewport main area (this will hold 3D and content later)
2. A minimal bottom nav bar for mobile with icons for:
   Explore, Launches, Timeline, Stats, Settings
3. A collapsible sidebar icon nav for desktop with the same items
4. A floating search bar component (command palette style) that
   opens with Cmd+K / Ctrl+K — just the UI shell for now, no
   search logic yet
5. Fully responsive — test at 375px, 768px, and 1440px widths

Use our design tokens for all colors, spacing, and typography.
No placeholder content — just the structural shell.
```

Review: Resize the browser across breakpoints. Test keyboard navigation (Tab through all nav items). Verify Cmd+K opens the search palette.

Step 3: Launch Database API & Data Layer

When: After layout shell works.

```
Referring to Section 5.1 of public/space-website-design-guide.md,
set up the launch database data layer:

1. Define the data model/schema for a launch record with these fields:
   - id, mission_name, launch_date, rocket_name, rocket_variant,
     provider, outcome (success/failure/partial/anomaly),
     orbit_type, launch_site, payload_type, payload_mass_kg,
     mission_description, mission_patch_url, booster_landing_outcome
2. Create a paginated API endpoint (or query function) that supports:
   - Cursor-based pagination (20 items per page)
   - Sort by date (default: newest first), name, provider
   - Filter by: provider, rocket, outcome, orbit_type, year range,
     launch_site
   - Full-text search across mission_name, rocket_name, provider,
     and mission_description
3. Seed it with sample data — at least 50 realistic launch records
   spanning multiple providers, years, and outcomes

Don't build the UI yet — just the data layer and API.
```

Review: Test the API/query functions directly — verify pagination works, filters return correct results, search finds partial matches.

Step 4: Launch Database Table/Card View

When: After API/data layer is tested and working.

```
Referring to Section 5.2 and 5.3 of public/space-website-design-guide.md,
build the Launches page with the database view:

DESKTOP (768px+):
1. A sortable data table with columns: Date, Mission, Rocket, Provider,
   Outcome (color-coded icon + text label), Orbit
2. Sticky column headers
3. Row hover state with subtle glow effect
4. Click a row to expand an inline detail panel showing mission
   description, payload info, and mission patch image
5. A "Columns" dropdown to show/hide optional columns (launch site,
   payload mass, booster landing)
6. Row density toggle: compact / comfortable / spacious

MOBILE (below 768px):
1. Card layout instead of table — each card shows mission name, date,
   rocket, provider, outcome badge
2. Tap to expand card with full details

BOTH:
1. Page controls showing "Page X of Y — Z total launches"
2. Outcome colors from our design tokens: green=success, red=failure,
   amber=partial — each with an icon (checkmark, X, warning) and text label
3. All using our design tokens. No hardcoded colors or font sizes.
```

Review: Verify table sorts correctly on each column. Test filter combinations. Check mobile card layout. Confirm color-coded outcomes also have icons and labels (accessibility). Screen reader test if possible.

Step 5: Search & Filter System

When: After the database view is rendering correctly.

```
Referring to Section 5.2 of public/space-website-design-guide.md,
wire up the search and filter system:

1. Connect the command palette (Cmd+K) to the database — live search
   results as the user types, with debounce (300ms). Show results
   grouped by category: Launches, Rockets, Providers.
2. Build a filter panel:
   - Desktop: collapsible sidebar on the Launches page
   - Mobile: bottom sheet that slides up from a "Filter" button
   - Filters: provider checkboxes, outcome checkboxes, year range
     slider, orbit type dropdown, launch site dropdown
3. Active filters shown as dismissible chips/tags above the table
4. "Clear all filters" button
5. URL state: filters and search terms should be reflected in the URL
   so users can share filtered views
6. Remember recent searches (last 5) in localStorage and show them
   as suggestions when the search palette opens

Test with edge cases: no results found, single result, special
characters in search.
```

Review: Test the full search flow — type queries, verify live results, click a result. Apply multiple filters simultaneously. Copy the URL, open in a new tab, confirm filters persist. Test the "no results" state.

Step 6: Basic 3D Star Field Background

When: After the database features are solid. This is the first visual enhancement.

```
Referring to Section 4 of public/space-website-design-guide.md,
add a basic 3D star field background to the site:

1. A full-viewport star field using Three.js (or react-three-fiber if
   we're in React) with subtle parallax movement on mouse move /
   device tilt
2. Stars should be instanced particles (InstancedMesh or Points) —
   aim for 2000-5000 stars with varying sizes and slight brightness
   variation
3. Very slow rotation so it feels alive even when the user isn't
   interacting
4. This renders BEHIND all page content as a background layer
5. Lazy-load the Three.js bundle — show a CSS gradient placeholder
   (#0a0a0f to #0b1528) until the 3D initializes
6. Set pixel ratio cap: Math.min(window.devicePixelRatio, 2)
7. Respect prefers-reduced-motion — if enabled, show a static star
   field image or CSS gradient instead
8. On mobile devices with less than 4GB RAM (estimate via
   navigator.deviceMemory), skip the 3D entirely and use the CSS
   gradient background

This should have ZERO impact on page interactivity — render on a
separate layer, ideally using OffscreenCanvas if browser supports it.
```

Review: Check FPS on desktop and mobile (use browser DevTools performance panel). Verify the fallback works by enabling `prefers-reduced-motion` in DevTools. Confirm the star field doesn't block clicks on any UI elements above it.

Step 7: Accessibility Audit

When: After Phase 1 features are complete. Do NOT skip this.

```
Referring to Section 9 of public/space-website-design-guide.md,
do a full accessibility audit and fix any issues:

1. Tab through every interactive element on every page — verify
   visible focus indicators (glowing ring using our accent color)
   on all buttons, links, inputs, table rows, and nav items
2. Verify all images have descriptive alt text
3. Verify the data table uses proper semantic HTML (table, thead,
   th with scope, tbody, td)
4. Verify every color-coded element (outcome badges) also has a
   text label and icon — never color alone
5. Add aria-labels to icon-only buttons (nav icons, filter toggle,
   density toggle)
6. Add aria-live region for search results count so screen readers
   announce "X results found" as user types
7. Test text scaling — set browser to 200% text size and fix any
   layout breakage
8. Verify prefers-reduced-motion disables all animations and the
   star field

List every issue found and fix them all in this session.
```

Review: Manually tab through the site yourself. Use a screen reader (VoiceOver on Mac, NVDA on Windows) for at least 5 minutes. Run Lighthouse accessibility audit.

PHASE 2: IMMERSION

Step 8: 3D Solar System View

When: After Phase 1 is fully tested and stable.

```
Referring to Sections 4.2 and 4.3 of public/space-website-design-guide.md,
build the interactive 3D solar system for the Explore page:

1. Sun at center with bloom/glow post-processing effect
2. All 8 planets with texture-mapped spheres orbiting at proportional
   (but compressed) distances. Include Earth's moon.
3. Orbit lines rendered as thin, semi-transparent rings
4. OrbitControls: click-drag to rotate, scroll to zoom, middle-click
   to pan. Center of rotation = currently focused object.
5. Click a planet to smoothly fly the camera to it (eased transition,
   ~1.5 seconds). Show a detail panel with planet info.
6. A "Reset View" button that smoothly returns to the default
   solar system overview
7. First-time visitor: show a brief animated hint overlay (hand icon
   showing drag-to-rotate, scroll-to-zoom) that dismisses on first
   interaction

PERFORMANCE:
- Use LOD: detailed planet models when close, simple spheres when
  far. Low-poly is fine for now.
- Use instancing for any repeated elements
- Lazy-load all planet textures — start with low-res (256px), swap
  to high-res (2048px) when user zooms into a specific planet
- Target: 60fps on desktop, 30fps on mobile
- Log renderer.info.render.calls — keep under 100

Provide a non-3D fallback for screen readers: an aria-label on the
canvas describing the current view, and ensure all planet info is
accessible via the database/text views too.
```

Review: Test on desktop (Chrome, Firefox, Safari) and mobile. Check FPS with DevTools. Test all planet clicks. Verify the fallback by disabling JavaScript or WebGL.

Step 9: Database-to-3D Integration

When: After the 3D solar system is working.

```
Referring to Sections 1 and 3.2 of public/space-website-design-guide.md
(the "dual-mode experience" and "contextual side panel" concepts):

Create a seamless bridge between the launch database and the 3D view:

1. Add a "View in 3D" button on each launch detail panel/card
2. Clicking it transitions to the Explore page and flies the 3D camera
   to the relevant launch site on Earth (Cape Canaveral, Vandenberg,
   Baikonur, etc.) with a marker/pin showing the launch location
3. The side panel stays open (or reopens) showing the launch details
   alongside the 3D view
4. Add launch site markers on the 3D Earth — small glowing dots at
   each major launch facility. Clicking one filters the database
   to show launches from that site.
5. Reverse direction: from the 3D solar system, clicking Earth should
   show launch site markers, and clicking a marker should open a
   filtered launch list for that site

The transition between database view and 3D view should be animated —
not a hard page reload. Use a shared layout or transition animation.
```

Review: Click "View in 3D" on several launches from different sites. Verify the camera flies to the correct location. Test the reverse flow (3D → database). Check that the URL updates to reflect the current state.

Step 10: Data Visualizations

When: After the database-to-3D bridge works.

```
Referring to Section 5.4 of public/space-website-design-guide.md,
build the Statistics page with interactive data visualizations:

1. LAUNCH FREQUENCY TIMELINE: Bar chart showing launches per year,
   stacked/colored by provider. Hover shows exact count. Click a
   bar to filter the database to that year.

2. SUCCESS RATE OVER TIME: Line chart showing rolling success rate
   by decade. Annotate major failure events (Challenger, Columbia,
   CRS-7) with markers.

3. LAUNCH SITE MAP: An interactive 2D world map (not the 3D globe —
   this is for quick data consumption) with proportionally-sized
   markers at each launch site. Hover for site name and launch count.
   Click to filter.

4. ROCKET COMPARISON: Side-by-side cards for top 5 most-launched
   rockets showing height (proportional illustration), total launches,
   success rate, and active/retired status.

All charts should:
- Use our design token colors
- Be interactive (hover tooltips, click to filter database)
- Be responsive (reflow gracefully on mobile)
- Include proper aria-labels and a text summary for screen readers
- Animate on scroll-into-view (respect prefers-reduced-motion)

Use a charting library that supports our needs — Recharts, Chart.js,
D3, or whatever best fits our stack.
```

Review: Test every chart interaction — hover, click, filter. Test on mobile. Verify the charts are readable by screen readers (at minimum, each chart should have an aria-label summarizing the key insight).

Step 11: Scrollytelling Homepage

When: After data visualizations are complete.

```
Referring to Section 6 of public/space-website-design-guide.md,
redesign the homepage as a scrollytelling experience:

HERO SECTION (viewport 1):
- Full-screen 3D star field with the site title in large display
  typography, a subtitle ("Every rocket launch in history,
  visualized"), and a "Start Exploring" button
- Subtle scroll indicator (animated chevron or "scroll" text) at bottom

SECTION 2 — THE NUMBERS:
- As user scrolls, large animated counters appear: total launches,
  active rockets, launch providers, countries. Numbers count up
  from 0 using eased animation.

SECTION 3 — TIMELINE OVERVIEW:
- A horizontal or vertical timeline showing key eras:
  Space Race (1957-1975), Shuttle Era (1981-2011),
  Commercial Revolution (2010-present)
- Each era has a brief description, a key stat, and a featured
  launch image/illustration
- The 3D background subtly transitions (e.g., star density changes,
  or Earth rotates into view)

SECTION 4 — FEATURED LAUNCHES:
- 3-4 cards highlighting historically significant missions
  (Apollo 11, first Shuttle, Falcon Heavy demo, etc.)
- Cards animate in on scroll with staggered timing

SECTION 5 — CTA:
- "Explore the Database" and "Enter the Solar System" buttons
  leading to the Launches and Explore pages

Use Intersection Observer for scroll detection. Ensure all content is
visible and readable even without JavaScript (progressive enhancement).
Respect prefers-reduced-motion — replace all animations with simple
fade-ins.
```

Review: Scroll through the entire homepage slowly and quickly. Test on mobile. Enable `prefers-reduced-motion` and verify all animations simplify. Disable JavaScript and verify content is still readable.

Step 12: Light Mode & Reduced Motion Polish

When: After the scrollytelling homepage is done.

```
Referring to Sections 2.2 and 9.1 of public/space-website-design-guide.md:

1. Build a light mode theme:
   - Invert the color system: light gray/white backgrounds, dark text
   - Adjust accent colors for readability on light backgrounds
     (some colors that work on dark fail on light — test each one)
   - The 3D star field should switch to a lighter style or be
     replaced with a subtle gradient
   - Add a theme toggle in Settings and in the nav (sun/moon icon)
   - Persist the user's choice in localStorage
   - Default to system preference via prefers-color-scheme media query

2. Polish the reduced-motion experience:
   - Every single animation on the site should have a reduced-motion
     alternative — audit them all
   - The 3D views should show a static rendered image with a
     "Load interactive view" button
   - Scrollytelling sections should use simple opacity fades only
   - Page transitions should be instant cuts, not animated

Test both modes thoroughly across all pages.
```

Review: Toggle between dark and light modes on every page. Check contrast ratios in light mode. Test the full reduced-motion experience end to end.

PHASE 3: ENGAGEMENT & POLISH

Step 13: Micro-Interactions

When: After Phase 2 is stable.

```
Referring to Section 7 of public/space-website-design-guide.md,
add micro-interactions throughout the site:

1. BUTTONS: Subtle radial glow behind buttons on hover (~200ms ease-in).
   Slight scale-up (1.02) on press, scale back on release.

2. LOADING STATES: Replace any spinners with themed animations —
   a small orbiting dot, a pulsing star, or a rocket silhouette
   with exhaust particles. Keep them tiny and fast.

3. DATA TABLE: Rows get a subtle scanning glow effect on hover —
   a horizontal light sweep across the row.

4. SEARCH: Results fade in with a slight upward slide (50ms stagger
   between items).

5. FILTER CHIPS: When added, chips scale up from 0 with a spring
   easing. When removed, they fade and shrink out.

6. TOAST NOTIFICATIONS: Slide in from top with a thin border that
   fills like a progress bar before auto-dismiss (5 seconds).

7. NAVIGATION: Active nav item has a glowing indicator dot/line.
   Page transitions use a brief fade-through-black (200ms).

8. SCROLL PROGRESS: A thin accent-colored line at the top of the
   viewport showing scroll progress on long pages.

ALL animations must:
- Use CSS transforms and opacity only (GPU-accelerated)
- Have durations from the design tokens (100-400ms range)
- Be disabled when prefers-reduced-motion is active
- Not block or delay user interaction
```

Review: Navigate every page, interact with every element. Verify animations feel snappy, not sluggish. Check FPS isn't impacted. Toggle reduced-motion to confirm all are disabled.

Step 14: Gamification Layer

When: After micro-interactions are polished.

```
Referring to Section 8 of public/space-website-design-guide.md,
add an optional gamification layer:

1. EXPLORATION TRACKER: Track which launches, rockets, providers,
   and planets the user has viewed. Store in localStorage. Show
   progress on a "Mission Log" page: "You've explored 47 of 7,476
   launches" with a progress bar.

2. ACHIEVEMENTS (mission patch badges):
   - "First Contact" — viewed your first launch
   - "Globe Trotter" — viewed launches from 5 different countries
   - "History Buff" — viewed a launch from before 1970
   - "Fleet Commander" — viewed all launches from one provider
   - "Solar Tourist" — visited every planet in the 3D solar system
   - Show a subtle toast notification when earned
   - Display earned badges on the Mission Log page

3. NEXT LAUNCH COUNTDOWN: A persistent but non-intrusive countdown
   widget (small, in the nav or footer) showing time until the next
   scheduled launch. Use a real data source if available, or a
   placeholder with a "Source: rocketlaunch.org" link.

4. DAILY FEATURED LAUNCH: On each visit, highlight a random
   historical launch as "Today's Mission" on the homepage or
   dashboard with a brief description and "Explore" button.

All gamification must be:
- Entirely optional — no content gated behind it
- Non-intrusive — no pop-ups blocking the UI
- Functional without an account (localStorage only)
```

Review: Clear localStorage and navigate the site fresh — verify achievements trigger correctly. Check that the gamification elements don't interfere with core functionality.

Step 15: Performance Optimization Pass

When: After all features are built. This is a dedicated optimization step.

```
Referring to Section 10 of public/space-website-design-guide.md,
do a comprehensive performance optimization pass:

1. Run Lighthouse on every page. Target scores:
   - Performance: 90+
   - Accessibility: 95+
   - Best Practices: 95+
   - SEO: 90+

2. CODE SPLITTING: Verify the 3D engine, charting library, and
   scrollytelling logic are in separate lazy-loaded chunks. The
   initial bundle should contain only the layout shell, theme,
   and routing.

3. ASSET OPTIMIZATION:
   - All 3D models compressed with Draco
   - All textures in KTX2 or WebP format
   - All images responsive (srcset with multiple sizes)
   - Font files subsetted to only used characters

4. CACHING: Set up a service worker that caches:
   - All static assets (models, textures, fonts, icons)
   - The app shell (HTML, CSS, core JS)
   - Recent API responses (launch data) with a stale-while-revalidate strategy

5. CORE WEB VITALS:
   - LCP under 2.5 seconds (ensure 3D canvas is NOT the LCP element)
   - INP under 200ms (verify 3D rendering doesn't block main thread)
   - CLS under 0.1 (all images and canvases have explicit dimensions)

6. MOBILE PERFORMANCE:
   - Test on a throttled connection (Slow 3G in DevTools)
   - Total initial page weight under 1MB
   - Time to interactive under 3 seconds

Fix every issue found. List what was changed and the before/after
Lighthouse scores.
```

Review: Run Lighthouse yourself and verify the scores. Test on a real mobile device if possible (not just DevTools emulation). Test on a slow WiFi connection.

Step 16: Final Cross-Browser & Device Testing

When: After performance optimization. This is the final step.

```
Do a final testing pass across browsers and devices. For each page
(Home, Explore/3D, Launches, Stats, Mission Log), verify:

BROWSERS:
- Chrome (latest)
- Firefox (latest)
- Safari (latest — especially test WebGL/WebGPU fallbacks)
- Edge (latest)
- Mobile Safari (iOS)
- Mobile Chrome (Android)

CHECK:
- Layout renders correctly with no overflow or alignment issues
- All interactions work (search, filters, 3D navigation, scrollytelling)
- Dark and light modes both look correct
- Animations play smoothly (or are disabled with reduced-motion)
- No console errors or warnings
- 3D fallback works when WebGL is unavailable

Fix any cross-browser issues found. List each issue and its fix.
```

Review: Test each browser yourself if you can. Pay special attention to Safari, which is the most common source of WebGL and CSS compatibility issues.

Post-Launch Ongoing Prompts

Once the overhaul is complete, here are prompts for future sessions:

Adding real launch data:

```
Read public/space-website-design-guide.md for context. I want to
replace our sample data with real launch records from [data source].
Import and normalize the data to match our schema from Step 3.
```

Adding a new scrollytelling mission deep-dive:

```
Read Section 6 of public/space-website-design-guide.md. Create a
scrollytelling deep-dive page for the [Apollo 11 / Falcon Heavy / etc.]
mission following the same patterns as the homepage scrollytelling.
```

Bug fixes:

```
Read public/space-website-design-guide.md for our design standards.
[Describe the bug]. Fix it while maintaining consistency with our
design tokens and accessibility requirements.
```

Timeline Expectations

Phase	Steps	Estimated Sessions
Phase 1: Foundation	Steps 0-7	4-6 sessions
Phase 2: Immersion	Steps 8-12	5-8 sessions
Phase 3: Polish	Steps 13-16	3-5 sessions
Total	**17 steps**	**12-19 sessions**

Each "session" is roughly one focused Claude Code conversation on a single step. Some complex steps (like Step 8: 3D Solar System) may need 2-3 sessions. Simple steps (like Step 6: Star Field) may be done in one.

The most important rule: Don't rush. Test each step thoroughly before moving on. Fixing a broken foundation in Phase 3 is ten times harder than getting it right in Phase 1.